LTV計算ロジックと検証ガイド

このドキュメントでは、`entry_product_ltv_by_month_offset` テーブルで算出されるLTV値の計算ロジックと、人間が手動で検証する方法を説明します。

目次

1. LTVテーブルの構造
2. 計算ロジックの概要
3. 詳細な計算手順
4. 検証用SQLクエリ
5. 計算例（サンプルデータ）

1. LTVテーブルの構造

テーブル名

`entry_product_ltv_by_month_offset`

主要カラム

カラム名	型	説明	計算方法
`entry_manage_number`	STRING	入口商品管理番号	初回注文で subtotal 最大の商品
`entry_item_name`	STRING	入口商品名	product_master 優先、なければ item_name
`first_order_month`	DATE	初回購入月（コホート）	初回注文日の月初（JST）
`month_offset`	INT64	経過月数	0=初月、1=1ヶ月後...
`cohort_users`	INT64	コホート人数	入口商品×初回購入月のユニークユーザー数
`active_buyers`	INT64	該当月の購入者数	該当月に購入があったユニークユーザー数
`revenue_in_month`	NUMERIC	該当月の売上	該当月の total_price 合計
`cumulative_revenue`	NUMERIC	累計売上	初月から該当月までの売上合計
`ltv_per_user`	NUMERIC	1人あたりLTV	累計売上 ÷ コホート人数
`aov_in_month`	NUMERIC	該当月の購入単価	該当月の売上 ÷ 該当月の購入者数

2. 計算ロジックの概要

処理フロー

```
orders + order_items
  ↓
① 各ユーザーの初回購入情報を特定
  ↓
② 初回購入時の「入口商品」を決定（subtotal最大）
  ↓
③ user_first_purchase_info テーブルに保存
  ↓
④ 全ユーザーの月別売上を集計
  ↓
⑤ 入口商品×初回購入月×経過月でグループ化
  ↓
⑥ LTV指標を計算（累計・1人あたり等）
```

重要な前提条件

- **キャンセル注文は除外**: `order_status != '900'`
- **月はJST基準**: `DATE(order_datetime, "Asia/Tokyo")`
- **累計は初月（month_offset=0）から**: `DATE_DIFF(order_month, first_order_month, MONTH) >= 0`

---

3. 詳細な計算手順

ステップ1: 初回購入情報の特定

```sql
-- 各ユーザーの初回注文を特定
WITH user_first_order AS (
  SELECT
    user_email,
    order_number AS first_order_number,
    DATE(order_datetime, "Asia/Tokyo") AS first_order_date,
    DATE_TRUNC(DATE(order_datetime, "Asia/Tokyo"), MONTH) AS first_order_month
  FROM (
    SELECT
      order_number,
      user_email,
      order_datetime,
      ROW_NUMBER() OVER(
        PARTITION BY user_email
        ORDER BY order_datetime ASC  -- 最も古い注文
      ) AS rn
    FROM `project.dataset.orders`
    WHERE order_status != '900'  -- キャンセル除外
  )
  WHERE rn = 1
)
```

確認ポイント:

- 1ユーザーにつき1行のみ（`user_email` でユニーク）
- `first_order_month` は月初日（例: 2024-01-15の注文 → 2024-01-01）

ステップ2: 入口商品の決定

```sql
-- 初回注文内で subtotal 最大の商品を「入口商品」とする
WITH entry_products AS (
  SELECT
    u.user_email,
    u.first_order_month,
    i.manage_number AS entry_manage_number,
    COALESCE(pm.product_name, i.item_name) AS entry_item_name,
    i.subtotal,
    ROW_NUMBER() OVER(
      PARTITION BY u.user_email
      ORDER BY i.subtotal DESC, i.manage_number ASC  -- subtotal降順、同額なら manage_number 昇順
    ) AS rn
  FROM user_first_order u
  JOIN `project.dataset.order_items` i
    ON i.order_number = u.first_order_number
  LEFT JOIN product_master pm
    ON pm.manage_number = i.manage_number
  WHERE i.manage_number IS NOT NULL
)
SELECT * FROM entry_products WHERE rn = 1
```

確認ポイント:

- 初回注文に複数商品がある場合、subtotal（小計）が最も大きいものを採用
- 同額の場合は `manage_number` の辞書順で最小のものを採用（一意性確保）

ステップ3: 月別売上の集計

```sql
-- 全ユーザーの月別売上を集計
WITH user_monthly_sales AS (
  SELECT
    user_email,
    DATE_TRUNC(DATE(order_datetime, "Asia/Tokyo"), MONTH) AS order_month,
    SUM(total_price) AS monthly_sales
  FROM `project.dataset.orders`
  WHERE order_status != '900'
  GROUP BY 1, 2
)
```

確認ポイント:

- 月単位で集計（月初日で丸める）
- `total_price` の合計（商品金額＋送料＋手数料等の総額）

ステップ4: 経過月数（month_offset）の計算

```sql
-- 初回購入月からの経過月数を計算
WITH user_cohort_sales AS (
  SELECT
    ufp.entry_manage_number,
    ufp.first_order_month,
    DATE_DIFF(ums.order_month, ufp.first_order_month, MONTH) AS month_offset,
    ums.user_email,
    ums.monthly_sales
  FROM user_first_purchase_info ufp
  JOIN user_monthly_sales ums
    ON ufp.user_email = ums.user_email
  WHERE DATE_DIFF(ums.order_month, ufp.first_order_month, MONTH) >= 0
)
```

month_offset の意味:

- `0`: 初回購入月と同じ月
- `1`: 初回購入の1ヶ月後
- `12`: 初回購入の12ヶ月後（1年後）

確認ポイント:

- `DATE_DIFF(月A, 月B, MONTH)` は月単位の差分
- 負の値（初回購入月より前）は除外

ステップ5: LTV指標の算出

```sql
-- コホート人数
WITH cohort_counts AS (
  SELECT
    entry_manage_number,
    first_order_month,
    COUNT(DISTINCT user_email) AS cohort_users
  FROM user_first_purchase_info
  GROUP BY 1, 2
),
-- 月別集計
monthly_aggregated AS (
  SELECT
    ucs.entry_manage_number,
    ucs.first_order_month,
    ucs.month_offset,
    cc.cohort_users,
    COUNT(DISTINCT ucs.user_email) AS active_buyers,
    SUM(ucs.monthly_sales) AS revenue_in_month
  FROM user_cohort_sales ucs
  JOIN cohort_counts cc
    ON ucs.entry_manage_number = cc.entry_manage_number
    AND ucs.first_order_month = cc.first_order_month
  GROUP BY 1, 2, 3, 4
)
-- 最終的な指標
SELECT
  entry_manage_number,
  first_order_month,
  month_offset,
  cohort_users,
  active_buyers,
  revenue_in_month,
  -- ウィンドウ関数で累計を計算
  SUM(revenue_in_month) OVER (
    PARTITION BY entry_manage_number, first_order_month
    ORDER BY month_offset
  ) AS cumulative_revenue,
  -- 1人あたりLTV
  SAFE_DIVIDE(
    SUM(revenue_in_month) OVER (
      PARTITION BY entry_manage_number, first_order_month
      ORDER BY month_offset
    ),
    cohort_users
  ) AS ltv_per_user,
  -- 該当月の購入単価
  SAFE_DIVIDE(revenue_in_month, NULLIF(active_buyers, 0)) AS aov_in_month
FROM monthly_aggregated
WHERE active_buyers > 0 OR revenue_in_month > 0
```

計算式まとめ:

- **累計売上** = `SUM(revenue_in_month)` を month_offset でソートしてウィンドウ関数で累積
- **1人あたりLTV** = 累計売上 ÷ コホート人数
- **該当月の購入単価** = 該当月の売上 ÷ 該当月の購入者数

---

4. 検証用SQLクエリ

4-1. 特定ユーザーの初回購入情報を確認

```
-- ユーザー "user@example.com" の初回購入情報
SELECT *
FROM `{project}.{dataset}.user_first_purchase_info`
WHERE user_email = 'user@example.com';
```

確認項目:

- `first_order_month` が実際の初回注文日の月初と一致しているか
- `entry_manage_number` が初回注文内で最大 subtotal の商品か

4-2. 特定商品×コホートのLTV推移を確認

```
-- 商品 "ABC123" で 2024年1月にデビューしたコホートのLTV推移
SELECT
  month_offset,
  cohort_users,
  active_buyers,
  revenue_in_month,
  cumulative_revenue,
  ltv_per_user,
  ROUND(ltv_per_user, 2) AS ltv_rounded
FROM `{project}.{dataset}.entry_product_ltv_by_month_offset`
WHERE entry_manage_number = 'ABC123'
  AND first_order_month = '2024-01-01'
ORDER BY month_offset;
```

確認項目:

- `cumulative_revenue` が単調増加しているか
- `month_offset=0` の `revenue_in_month` が初月売上と一致するか
- `ltv_per_user = cumulative_revenue / cohort_users` の計算が合っているか

4-3. 手動で累計売上を再計算して検証

```sql
-- 特定コホートの累計売上を手動で計算
WITH manual_calc AS (
  SELECT
    month_offset,
    revenue_in_month,
    SUM(revenue_in_month) OVER (ORDER BY month_offset) AS manual_cumulative
  FROM `{project}.{dataset}.entry_product_ltv_by_month_offset`
  WHERE entry_manage_number = 'ABC123'
    AND first_order_month = '2024-01-01'
)
SELECT
  month_offset,
  revenue_in_month,
  manual_cumulative,
  cumulative_revenue AS table_cumulative,
  -- 差分確認
  ABS(manual_cumulative - cumulative_revenue) AS diff
FROM manual_calc
JOIN `{project}.{dataset}.entry_product_ltv_by_month_offset` USING (month_offset)
WHERE entry_manage_number = 'ABC123'
  AND first_order_month = '2024-01-01'
ORDER BY month_offset;
```

確認項目:

- `diff` がすべて 0 であれば整合性OK

4-4. コホート人数の検証

```sql
-- テーブルに記録されたコホート人数
SELECT
  entry_manage_number,
  first_order_month,
  cohort_users
FROM `{project}.{dataset}.entry_product_ltv_by_month_offset`
WHERE month_offset = 0  -- 初月のレコードから取得
  AND entry_manage_number = 'ABC123'
  AND first_order_month = '2024-01-01';

-- user_first_purchase_info から直接カウント（実数）
SELECT
  entry_manage_number,
  first_order_month,
  COUNT(DISTINCT user_email) AS actual_cohort_users
FROM `{project}.{dataset}.user_first_purchase_info`
WHERE entry_manage_number = 'ABC123'
  AND first_order_month = '2024-01-01'
GROUP BY 1, 2;
```

確認項目:

- 両方の `cohort_users` が一致しているか

4-5. 該当月の購入単価（AOV）の検証

```sql
-- テーブル値と手動計算の比較
SELECT
  month_offset,
  revenue_in_month,
  active_buyers,
  aov_in_month AS table_aov,
  SAFE_DIVIDE(revenue_in_month, NULLIF(active_buyers, 0)) AS manual_aov,
  ABS(aov_in_month - SAFE_DIVIDE(revenue_in_month, NULLIF(active_buyers, 0))) AS diff
FROM `{project}.{dataset}.entry_product_ltv_by_month_offset`
WHERE entry_manage_number = 'ABC123'
  AND first_order_month = '2024-01-01'
ORDER BY month_offset;
```

確認項目:

- `diff` が 0（または丸め誤差程度）であれば整合性OK

---

5. 計算例（サンプルデータ）

前提条件

- **入口商品**: `manage_number = "PROD001"`（商品A）
- **初回購入月**: 2024年1月（`first_order_month = 2024-01-01`）
- **コホート人数**: 10人

月別データ

month_offset	購入者数	該当月の売上	説明
0	10	100,000円	初月：全員が購入（初回購入）
1	3	30,000円	1ヶ月後：3人がリピート
2	2	25,000円	2ヶ月後：2人がリピート
3	0	0円	3ヶ月後：購入なし
4	1	15,000円	4ヶ月後：1人が購入

計算結果

month_offset	cohort_users	active_buyers	revenue_in_month	cumulative_revenue	ltv_per_user	aov_in_month
0	10	10	100,000	100,000	10,000	10,000
1	10	3	30,000	130,000	13,000	10,000
2	10	2	25,000	155,000	15,500	12,500
3	10	0	0	155,000	15,500	NULL
4	10	1	15,000	170,000	17,000	15,000

計算式の検証

month_offset = 2 の場合

- **累計売上**: 100,000 + 30,000 + 25,000 = **155,000円** ✓
- **1人あたりLTV**: 155,000 ÷ 10 = **15,500円** ✓
- **該当月の購入単価**: 25,000 ÷ 2 = **12,500円** ✓

month_offset = 3 の場合

- **累計売上**: 155,000 + 0 = **155,000円** ✓ （前月と同じ）
- **1人あたりLTV**: 155,000 ÷ 10 = **15,500円** ✓ （前月と同じ）
- **該当月の購入単価**: 0 ÷ 0 = **NULL** ✓ （購入者0のため）

> **注意**: `month_offset=3` のような「購入者0」の行は、実際のテーブルには保存されません（`WHERE active_buyers > 0 OR revenue_in_month > 0` で除外）。

---

6. よくある質問と確認ポイント

Q1. LTVが減少することはあるか？

A: ありません。`cumulative_revenue`（累計売上）は単調増加するため、`ltv_per_user` も単調増加します。もし減少していたら計算ミスです。

Q2. 同じ month_offset で複数行あることはあるか？

A: あります。`entry_manage_number`（入口商品）と `first_order_month`（コホート）の組み合わせごとに行が作られるためです。

Q3. month_offset に上限はあるか？

A: ありません。以前は 0〜12 の制限がありましたが、今回の実装で撤廃されました。

Q4. 「購入なし」の月は記録されるか？

A: 記録されません。`active_buyers=0` かつ `revenue_in_month=0` の行はテーブルから除外されます。

Q5. データの鮮度は？

A: 月次バッチ実行後、全期間が再計算されます。`updated_at` カラムで最終更新日時を確認できます。

---

7. トラブルシューティング

問題: LTV値が異常に大きい/小さい

確認手順:

1. `cohort_users` が正しいか（user_first_purchase_info と照合）
2. `revenue_in_month` が正しいか（orders の total_price 合計と照合）
3. キャンセル注文が除外されているか（`order_status != '900'`）

問題: 累計が前月より減っている

原因: 計算ロジックのバグ。ウィンドウ関数の `ORDER BY month_offset` が正しく動作していない可能性。

問題: 特定ユーザーがテーブルに反映されない

確認手順:

1. `user_first_purchase_info` にそのユーザーがいるか
2. 初回注文に `manage_number` があるか（NULL だと除外）
3. キャンセル注文ではないか

---

8. まとめ

このドキュメントで説明した検証SQLを使い、以下を定期的に確認してください：

- ☐ 累計売上が単調増加している
- ☐ 1人あたりLTV = 累計売上 ÷ コホート人数 が成立
- ☐ コホート人数が user_first_purchase_info と一致
- ☐ 該当月の購入単価 = 該当月の売上 ÷ 購入者数 が成立

異常値を発見した場合は、該当のコホート・月について上記の検証SQLで詳細を追跡してください。